

Laboratorio de Sistemas Operativos

Gestión de Memoria

Práctica No. 3

Fecha de inicio: Abr. 20 de 2026
Fecha de entrega: May. 03 de 2026 (23:59)
Medio de entrega: Moodle

Objetivo

Esta práctica evalúa los conceptos fundamentales de la visualización de memoria en sistemas operativos modernos. A través de programas en C y herramientas de Linux, el estudiante comprobará empíricamente los mecanismos que el SO utiliza para ofrecer a cada proceso la ilusión de un espacio de direcciones privado y contiguo.

*** Herramientas útiles ***

Compilador: `gcc -Wall -o nombre nombre.c`

Valgrind: `valgrind --leak-check=full ./nombre`

Sino cuenta con Valgrind, instalarlo: `sudo apt install valgrind`

1 Espacio de direcciones

Tema libro de texto: `vm-intro` — The Abstraction: Address Spaces

El SO crea para cada proceso la abstracción de un **espacio de direcciones virtual** privado, compuesto por tres regiones principales: código (*text*), *heap* y *stack*. Linux expone esta información en el pseudo-sistema de archivos **/proc**.

1.1 Actividad: Programa base

Guarde el siguiente código como `mem_map.c` y compilelo:

```
1 // mem_map.c
2 #include <stdio.h>
```

```
3  #include <stdlib.h>
4  #include <unistd.h>
5
6  int global_var = 42;           /* segmento de datos (.data) */
7
8  int main() {
9      int local_var = 10;        /* stack */
10     int *heap_var = malloc(sizeof(int)*100); /* heap */
11     *heap_var = 99;
12
13     printf("PID del proceso      : %d\n", getpid());
14     printf("Dir. codigo (main)   : %p\n", (void*)main);
15     printf("Dir. global_var      : %p\n", (void*)&global_var);
16     printf("Dir. local_var       : %p\n", (void*)&local_var);
17     printf("Dir. heap_var        : %p\n", (void*)heap_var);
18
19     printf("\nPresione ENTER para continuar...\n");
20     getchar();
21     free(heap_var);
22     return 0;
23 }
```

Code 1: mem_map.c

1.2 Actividad: Visualizar los mapas de memoria de un proceso

Mientras el programa espera el ENTER, abra una segunda terminal:

```
# Leer el mapa de memoria del proceso
cat /proc/$(pgrep mem_map)/maps

# Vista resumen con tamanos de cada region
pmap -x $(pgrep mem_map)
```

1.3 Actividad: Exploración de /proc/[pid]/maps

1. Identifique en la salida de /proc/maps las regiones *text*, *heap* y *stack*. ¿Qué permisos (r/w/x/p) tiene cada una? ¿Por qué difieren?
2. Compare las direcciones impresas con los rangos de /proc/maps. ¿A qué región pertenece cada variable?
3. ¿Qué otras regiones aparecen en el mapa (libc, [vdso], [vsyscall])? ¿Que funcion cumple cada una?

4. ¿Son las direcciones virtuales iguales a las físicas? Explique apoyandose en el concepto de *address space* del OSTEP.

1.4 Actividad: Comparar espacios de dos procesos simultáneos

Ejecute dos instancias de `mem_map` al mismo tiempo en dos terminales distintas y compare las direcciones de `main`, `global_var` y `heap_var`.

1. ¿Son las mismas direcciones virtuales en ambos procesos? ¿Qué conclusión saca sobre el aislamiento del espacio de direcciones?
2. ¿Podría el Proceso A leer o modificar la variable global del Proceso B mediante su dirección virtual? Justifique.

2 API de Memoria

Tema libro de texto: `vm-api` — Interlude: Memory API

En C la memoria del heap se administra explícitamente con `malloc()`, `calloc()`, `realloc()` y `free()`. Los errores mas comunes son: *memory leak* (no liberar), *double free* (liberar dos veces), *buffer overflow* (escribir fuera del bloque asignado) y *use-after-free* (acceder a memoria ya liberada). **Valgrind** detecta todos estos errores en tiempo de ejecución.

2.1 Actividad: Programa base

Cree el archivo `heap_demo.c` y compilelo:

```
1 // heap_demo.c
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main() {
6     int n = 10;
7     int *arr = (int *) malloc(n * sizeof(int));
8     if (arr == NULL) { perror("malloc"); return 1; }
9
10    for (int i = 0; i < n; i++) arr[i] = i * i;
11    printf("Arreglo original: ");
12    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
13    printf("\n");
14
15    /* Redimensionar a 20 enteros */
16    arr = (int *) realloc(arr, 20 * sizeof(int));
17    if (arr == NULL) { perror("realloc"); return 1; }
18    for (int i = n; i < 20; i++) arr[i] = i * i;
```

```
19     printf("Arreglo ampliado: ");
20     for (int i = 0; i < 20; i++) printf("%d ", arr[i]);
21     printf("\n");
22
23     free(arr);
24     return 0;
25 }
26
```

Code 2: heap_demo.c

Una vez compilado, monitoree la ejecución del programa con Valgrind:

```
gcc -Wall -o heap_demo heap_demo.c
valgrind --leak-check=full --track-origins=yes ./heap_demo
```

2.2 Actividad: Uso correcto de **malloc** y **free**

1. Muestre la salida completa de Valgrind. ¿Reporta errores o fugas de memoria? ¿Que significa el mensaje "All heap blocks were freed"?
2. ¿Por qué se usa `sizeof(int)` en lugar del valor literal 4? ¿Qué ventaja ofrece en portabilidad entre arquitecturas?
3. ¿Qué devuelve `malloc` cuando no hay memoria disponible? ¿Por qué es crítico verificar ese valor antes de usarlo?

2.3 Actividad: Código con *bugs* de memoria

El siguiente código tiene **tres errores clásicos**. Guarde como `buggy_mem.c`:

```
1 // buggy_mem.c -- NO ejecutar sin Valgrind
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 int main() {
7     /* ERROR 1: buffer overflow */
8     int *p = malloc(5 * sizeof(int));
9     for (int i = 0; i <= 5; i++) /* <= en vez de < */
10        p[i] = i;
11
12     /* ERROR 2: memory leak (nunca se llama free(q)) */
13     char *q = malloc(100);
14     strcpy(q, "hola mundo");
15     printf("%s\n", q);
16 }
```

```
17      /* ERROR 3: use-after-free */
18      free(p);
19      printf("p[0] = %d\n", p[0]);    /* acceso ilegal */
20
21      return 0;
22  }
```

Code 3: buggy_mem.c — contiene errores intencionales

Compile y ejecute el programa:

```
gcc -Wall -g -o buggy_mem buggy_mem.c
valgrind --leak-check=full --track-origins=yes ./buggy_mem
```

2.4 Actividad: Identificar y corregir errores de memoria

1. Transcriba los mensajes que arroja Valgrind. ¿Cual mensaje corresponde a cada uno de los tres errores clásicos?
2. Corrija el programa (buggy_mem_fixed.c) y verifique con Valgrind que no queda ningún error ni fuga.
3. ¿Qué consecuencias puede tener un *use-after-free* en un programa real en términos de seguridad y estabilidad del sistema?

3 Traducción de direcciones — Base & Bounds

Tema del libro: vm-mechanism — Mechanism: Address Translation

El mecanismo mas simple de traducción de direcciones utiliza dos registros de hardware: **base** (dirección física donde inicia el proceso) y **bounds** (tamaño del espacio asignado). La formula de traducción es:

$$PA = VA + base, \quad \text{valida solo si } 0 \leq VA < bounds$$

Si la condición no se cumple, el hardware lanza una excepción que el SO atiende como una violación de segmento (*segmentation fault*).

3.1 Actividad: Simulador

Guarde como `base_bounds.c`, aquí **VA** hace referencia a la dirección virtual y **PA** a la dirección física:

```
1  // base_bounds.c
2  #include <stdio.h>
3
4  typedef struct { int base; int bounds; } Registro;
```

```
5
6  /* Traduce VA -> PA; imprime excepcion si viola bounds */
7  int traducir(Registro r, int va) {
8      if (va < 0 || va >= r.bounds) {
9          printf("  [EXCEPCION] VA=%d viola bounds=%d\n",
10             va, r.bounds);
11          return -1;
12      }
13      return r.base + va;
14  }
15
16  int main() {
17      Registro procA = {32, 64}; /* base=32, bounds=64 */
18      Registro procB = {128, 80}; /* base=128, bounds=80 */
19      int vas[] = {0, 10, 63, 64, 100};
20      int n = sizeof(vas) / sizeof(vas[0]);
21
22      printf("--- Proceso A (base=%d, bounds=%d) ---\n",
23          procA.base, procA.bounds);
24      for (int i = 0; i < n; i++) {
25          int pa = traducir(procA, vas[i]);
26          if (pa != -1)
27              printf("  VA=%3d -> PA=%3d\n", vas[i], pa);
28      }
29      printf("--- Proceso B (base=%d, bounds=%d) ---\n",
30          procB.base, procB.bounds);
31      for (int i = 0; i < n; i++) {
32          int pa = traducir(procB, vas[i]);
33          if (pa != -1)
34              printf("  VA=%3d -> PA=%3d\n", vas[i], pa);
35      }
36      return 0;
37  }
```

Code 4: base_bounds.c

3.2 Actividad: Base & Bounds — Análisis

1. Compile y ejecute. Muestre la salida completa. ¿Que ocurre al acceder a VA=64 y VA=100 en el Proceso A? ¿Que haria el SO real ante esta excepcion?
2. Agregue un Proceso C (base=0, bounds=32) al programa y traduzca las mismas VAs. ¿Puede el Proceso A acceder a las direcciones del Proceso C directamente? Justifique.
3. ¿Cual es la limitacion principal del esquema base & bounds que motiva el surgimiento de la segmentacion?

4 Segmentacion

Tema del libro: `vm-segmentation` — Segmentation

La segmentación divide el espacio de direcciones en segmentos lógicos independientes (*code*, *heap*, *stack*), cada uno con su propio par base/bounds. Los **bits mas significativos** de la VA identifican el segmento; los bits restantes forman el **offset**.

4.1 Traducción manual con tabla de segmentos

Dado un espacio de **14 bits** (2 bits de selector + 12 bits de offset), use la siguiente tabla de segmentos:

Segmento	Base (PA)	Tamaño	Crece	Selector
Code	0x4000 (16 384 B)	2 KB	positivo →	00
Heap	0x6000 (24 576 B)	3 KB	positivo →	01
Stack	0x2800 (10 240 B)	2 KB	negativo ←	11

Traduzca manualmente y complete la tabla:

VA (hex)	Selector	Offset	Segmento	PA o Excepcion
0x03A0	00	0x3A0	Code	
0x1800	01	0x800	Heap	
0x3C00	11	0xC00	Stack	
0x0C00	00	0xC00	Code	
0x2200	10	—	???	

1. Muestre el cálculo paso a paso para cada VA.
2. ¿Por qué el Stack crece en dirección negativa? ¿Que ajuste especial requiere la formula al calcular el PA?
3. ¿Qué ventaja tiene la segmentación frente a base & bounds en cuanto a utilizacion de la memoria física?
4. ¿Qué es la *fragmentación externa*? ¿Por que surge con segmentación? Ilustre con un diagrama de bloques de memoria.

5 Paginación

Tema del libro: vm-paging — Introduction to Paging

La paginación divide el espacio virtual en **páginas** de tamaño fijo y la memoria física en **marcos** (*frames*) del mismo tamaño. Una **tabla de páginas** mapea cada VPN al PFN correspondiente. La descomposición de la VA es:

$$PA = \underbrace{PFN}_{\text{de la tabla}} \times \text{PageSize} + \underbrace{\text{offset}}_{\text{bits bajos de VA}}$$

Acá hay tres definiciones útiles:

5.0.1 VPN — Virtual Page Number (Número de Página Virtual)

Es la ‘dirección’ de una página dentro del espacio virtual del proceso. Cuando el proceso genera una dirección virtual, los bits más altos son el VPN y le dicen al hardware qué página está buscando. Es como el número de un apartamento en un edificio: identifica la unidad, pero no dice en qué piso físico está. Ejemplo: con páginas de 4 KB (12 bits de offset) y una VA de 32 bits, los 20 bits superiores son el VPN → hay hasta 1.048.576 páginas virtuales posibles.

5.0.2 PFN — Physical Frame Number (Número de Marco Físico)

Es el equivalente físico del VPN: identifica un bloque concreto de la RAM real. La memoria física se divide en “marcos” (frames) del mismo tamaño que las páginas, y cada marco tiene su número. El trabajo de la tabla de páginas es precisamente decir: “la página virtual número X está en el marco físico número Y”, es decir, VPN → PFN.

5.0.3 PTE — Page Table Entry (Entrada de la Tabla de Páginas)

Es una fila de la tabla de páginas. Cada página virtual tiene su propia PTE, y dentro de ella se almacenan dos cosas principales: El PFN (a qué marco físico corresponde esa página) y Varios bits de control con información útil:

5.1 Actividad: Cálculo de la tabla de páginas

Considere el sistema:

Parametro	Valor
Espacio virtual	32 bits
Tamano de pagina	4 KB = 2^{12} bytes
Espacio fisico	20 bits (1 MB)
Tamano de PTE	4 bytes

1. ¿Cuántos bits se necesitan para el VPN y cuántos para el offset? Muestre el cálculo.
2. ¿Cuántas entradas tiene la tabla de páginas de un proceso?
3. ¿Cuánta memoria ocupa la tabla de páginas completa? ¿Es razonable ese tamaño para cada proceso?
4. ¿Cuántos bits necesita el PFN dentro de la PTE? ¿Qué información almacenan los bits restantes? Mencione al menos **3 bits de control** y su función.

5.2 Actividad: Simulador de paginación

Guarde como `paging_sim.c`:

```
1 // paging_sim.c
2 #include <stdio.h>
3
4 #define PAGE_BITS 4
5 #define PAGE_SIZE (1 << PAGE_BITS) /* 16 bytes/pagina
6   */
7 #define VA_BITS 8 /* VA de 8 bits
8   */
9 #define NUM_PAGES (1 << (VA_BITS - PAGE_BITS)) /* 16 paginas
10   */
11
12 /* Tabla de paginas: -1 = pagina no presente (PAGE FAULT) */
13 int page_table[NUM_PAGES] = {
14     3, -1, 7, 2, -1, 1, -1, 5,
15     -1, -1, 4, -1, 6, -1, 0, -1
16 };
17
18 void traducir(int va) {
19     int vpn = va >> PAGE_BITS;
20     int offset = va & (PAGE_SIZE - 1);
21     printf("VA=0x%02X VPN=%2d Offset=%2d ", va, vpn, offset);
22     if (page_table[vpn] == -1) {
23         printf("-> PAGE FAULT (pagina no presente)\n");
24     } else {
25         int pfn = page_table[vpn];
26         int pa = (pfn << PAGE_BITS) | offset;
27         printf("-> PFN=%2d PA=0x%02X\n", pfn, pa);
28     }
29 }
30
31 int main() {
32     int vas[] = {0x00, 0x0F, 0x20, 0x35, 0x10, 0xA3, 0xC8, 0xF0};
33     int n = sizeof(vas) / sizeof(vas[0]);
```

```
31     printf("%-22s %-6s %-8s %-6s %s\n",
32           "VA", "VPN", "Offset", "PFN", "PA");
33     printf("-----\n"
34           );
35     for (int i = 0; i < n; i++) traducir(vas[i]);
36     return 0;
```

Code 5: paging_sim.c

5.3 Actividad: Simulador — Análisis

1. Compile y ejecute el simulador. Muestre la salida completa.
2. ¿Que ocurre con las VAs 0x10 y 0xA3? ¿Que debería hacer el SO real ante un *page fault*?
3. ¿Cuántos accesos a memoria física requiere completar una instrucción *load* con tabla de paginas de un solo nivel? ¿Por que es costoso y que solución de hardware existe?
4. ¿Que ventaja tiene la paginación sobre la segmentación en cuanto a la fragmentación?

Entrega y evaluación

Incluya un enlace a un repositorio de github con:

1. Informe donde se explica claramente la solución (README.md). El informe debe contar con las siguientes secciones:
 - (a) Nombres completos de los integrantes, correos y números de documento.
 - (b) Documentación de todas las funciones desarrolladas en el código.
 - (c) Problemas presentados durante el desarrollo de la práctica y sus soluciones.
 - (d) Pruebas realizadas a los programas que verificaron su funcionalidad.
 - (e) Un enlace a un video de 10 minutos donde se sustente el desarrollo.
 - (f) Manifiesto de transparencia: En que puntos se apoyaron de la IA generativa.
2. Los códigos fuentes de las aplicaciones en una carpeta aparte en el repositorio.

Trabajo a realizar en parejas, o en caso excepcional grupo de 3 personas, evitar trabajar solo.

Entregar un solo repositorio por grupo.